

Природа помилки: виняток чи альтернатива?



Традиційний погляд:
Помилка – це «аварія»
(Exception).



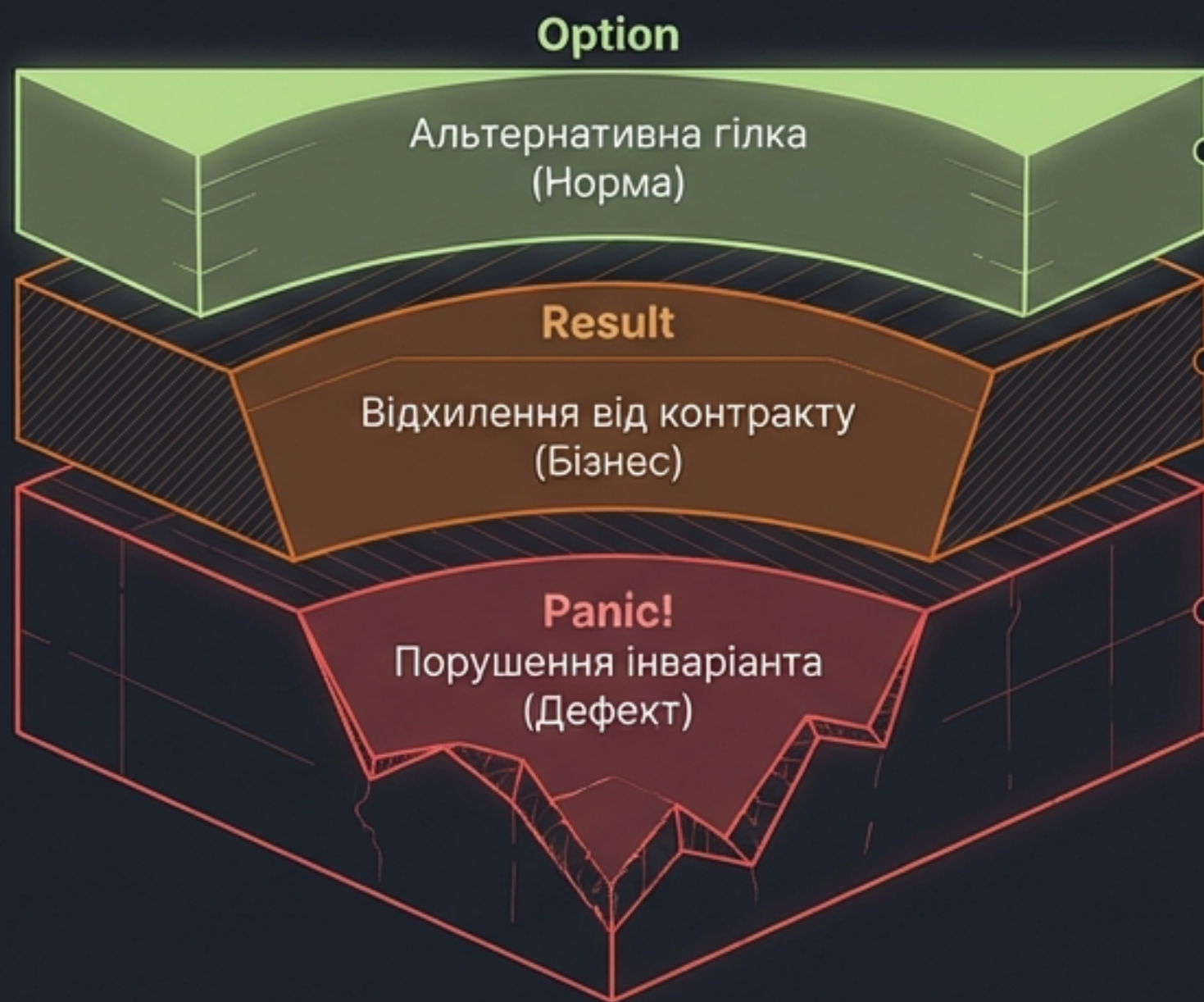
Погляд Rust:
Помилка – це дані
(Option/Result).

```
// Імперативний підхід (Java/C# style):  
// throw new UserNotFoundException();
```

```
// Підхід Rust (Алгебраїчний тип):  
fn find_user(id: i32) -> Option<User> {  
    // Явно повертаємо "Нічого", якщо  
    користувача немає  
    None  
}
```

`Option<T>` = Результат відсутній | `Result<T, E>` = Операція невдала

Помилка як частина контракту



```
struct User { id: i32, name: String }
```

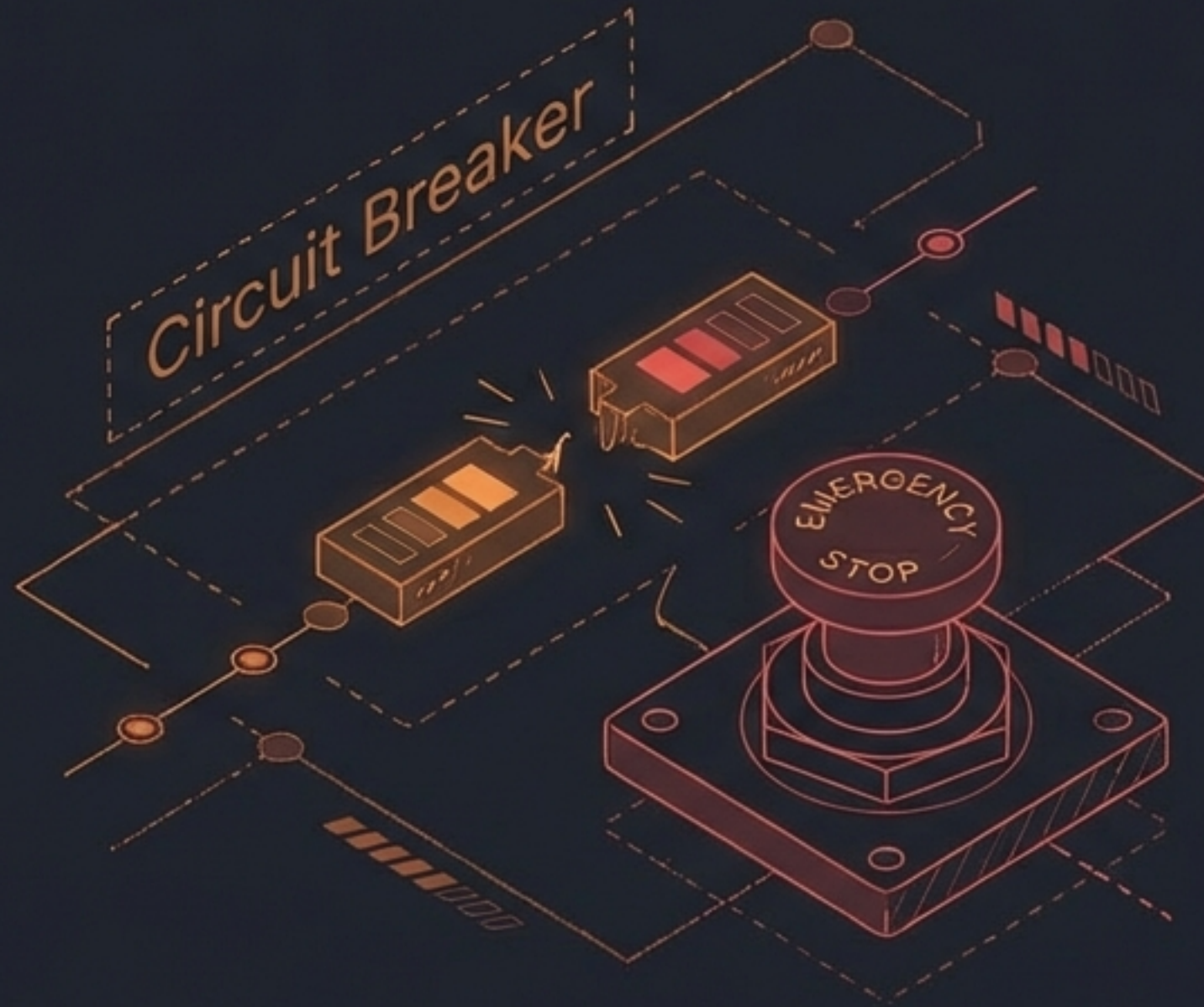
```
// Це не помилка, це альтернативний стан  
fn get_extension(filename: &str) -> Option<&str> {  
    filename.split('.').last()  
}
```

```
// Порушення інваріанта (Panic)  
fn divide(a: i32, b: i32) -> i32 {  
    if b == 0 { panic!("Division by zero is impossible!") }  
    a / b  
}
```

Чи всі помилки потрібно обробляти?

РЕПРЕЗЕНТАЦІЯ ≠ ВИПРАВЛЕННЯ

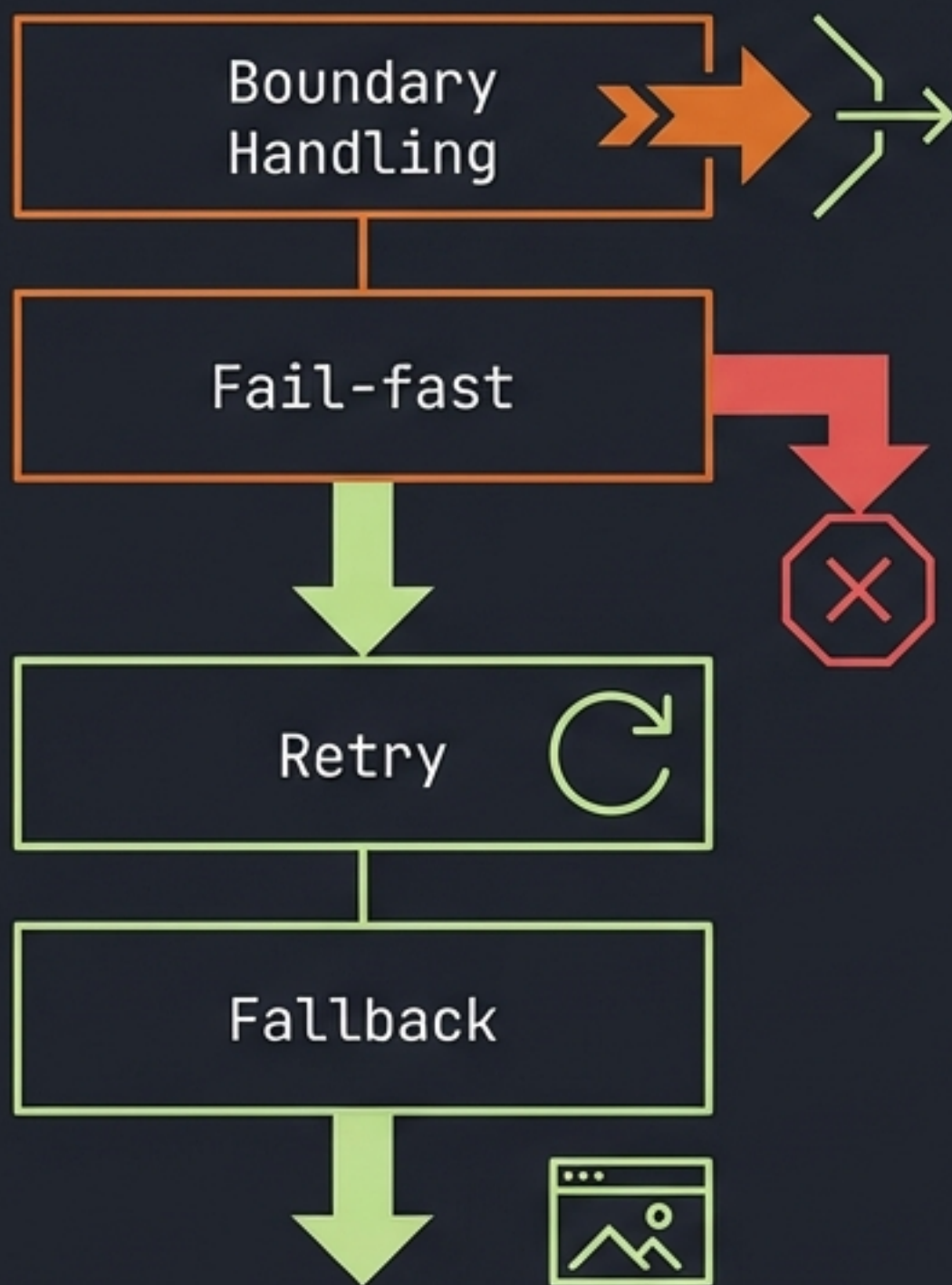
- Всі відхилення мають бути явними.
- Rust забороняє ігнорувати Result (compiler warning).



Стратегія Fail-fast

- Система в некоректному стані → Завершуйте роботу.
- Не маскуйте баги.

Стратегії роботи з помилками



```
fn main() -> Result<(), Box<dyn std::error::Error>> {  
    // Boundary handling: делегування помилки наверх  
    let config = load_config()?;  
  
    // Fail-fast логіка всередині  
    run_app(config)  
}
```

Класифікація помилок

ТИП	ПРИКЛАД	РЕАКЦІЯ
Логічні (Code Defects)	Index out of bounds	Fix code / Panic
Технічні (IO/Network)	Connection timeout	Retry / Result
Доменні (Business Rules)	Insufficient funds	Result (Explicit)
Інфраструктурні	DB is down	Circuit Breaker

Чітке розмежування дозволяє обрати правильний інструмент.

Доменні помилки: Приклад

Доменна помилка — це частина бізнес-процесу.

Вона має бути типізована.

```
enum PaymentError {  
    InsufficientFunds { current: f64, required: f64 },  
    CardExpired,  
    LimitExceeded,  
}  
  
fn process_payment(amount: f64) -> Result<(), PaymentError> {  
    if amount > 1000.0 {  
        return Err(PaymentError::LimitExceeded);  
    }  
    Ok(())  
}
```

← Типізована подія, а не краш

Помилка як частина контракту функції

Чесний контракт (Rust) ✓

```
fn open_file(path: &str) -> Result<File, io::Error>
```

Я обіцяю повернути `File` АБО `Error`.

Брехливий контракт (Traditional) ✗

```
File openFile(String path)
```

Я обіцяю **File**... але можу кинути **Exception** і вбити процес.

Тип функції — це її публічне зобов'язання.

Total vs Partial Functions



```
// Partial (Часткова) - Небезпечно
fn first_element(list: &Vec<i32>) -> i32 {
    list[0] // Паніка, якщо вектор порожній
}
```

```
// Total (Повна) - Безпечно
fn first_element_safe(list: &Vec<i32>) -> Option<i32> {
    list.get(0).copied() // Повертає None
}
```

Переваги Total Programming



Композиційність

Total функції легко поєднувати.



Тестування

Немає прихованих шляхів паніки.



Безпека

Відсутність runtime-помилки.



Повторне використання

Без знання внутрішньої реалізації.

Чому NULL — це не модель

“The Billion Dollar Mistake”

- Відсутність семантики
- Відсутність типового захисту
- Runtime Crashes (NPE)

```
// Інші мови:  
String s = null;  
➔ s.length(); // CRASH! ❗
```

```
// Rust:  
let s: String = "Hello".to_string();  
+ // Гарантовано має значення ✅
```

Option: Перемога над Null

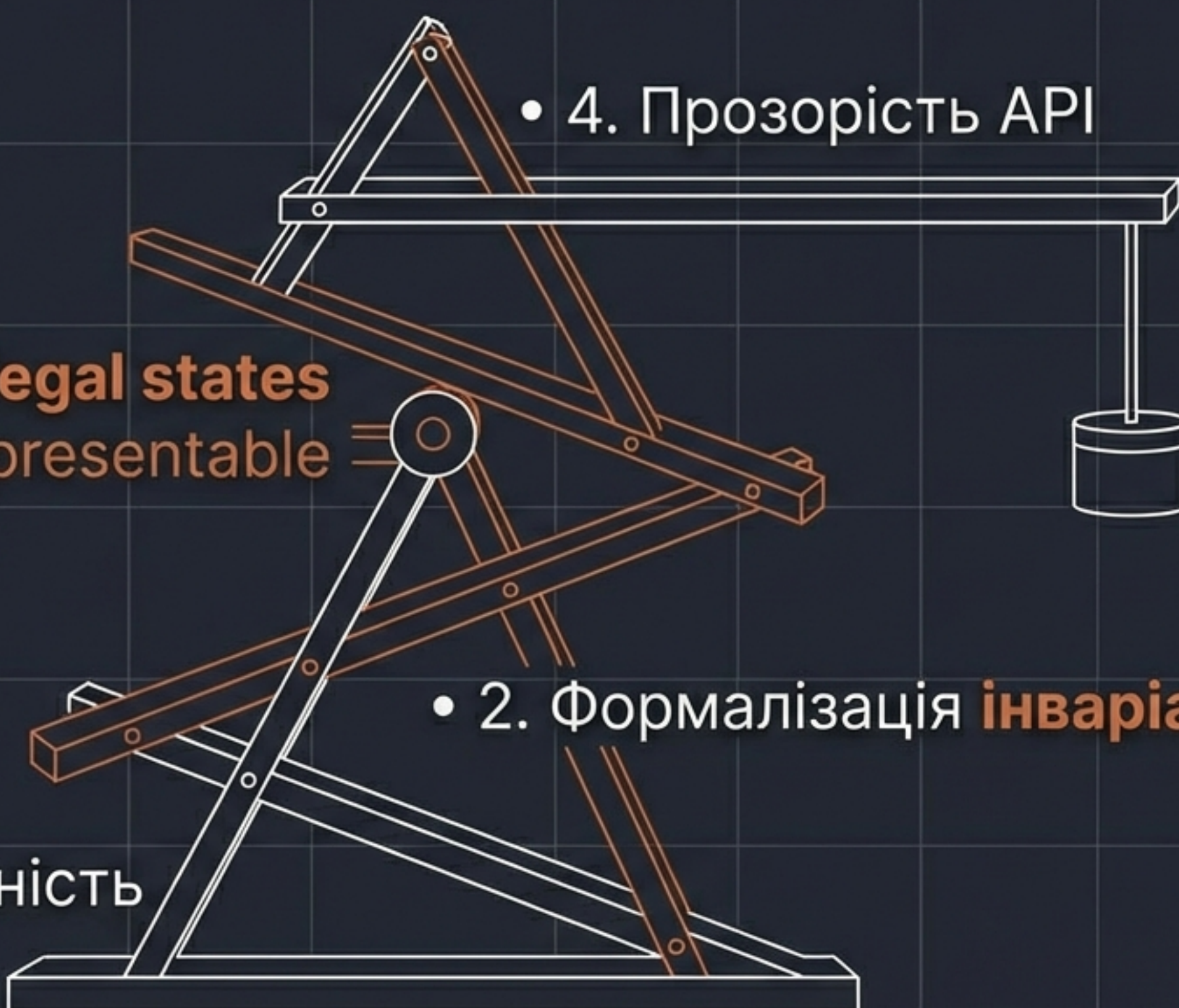
`Option<T>` переносить перевірку на етап компіляції. Ви зобов'язані розпакувати значення.

```
let name: Option<String> = get_name();

// Компілятор не дозволить:
// println!("{}", name.len()); // ✗ Error!

// Ви мусите обробити обидва варіанти:
match name {
    Some(n) => println!("Length: {}", n.len()),
    None => println!("Name is missing"),
}
```

Дисципліна мислення без Null

- 
- 1. Явна варіативність
 - 2. Формалізація **інваріантів**
 - 3. Make **illegal states** **unrepresentable**
 - 4. Прозорість API

Option<A>: Репрезентація відсутності



Search
(Collection lookup)



Cache (Hit/Miss)



Configuration
(Optional params)



Lazy Initialization

```
let config_val = config.get("timeout");  
// Option<&Val>  
  
struct UserArgs {  
    email: String,  
    age: Option<u8>, // Необов'язкове поле  
}
```

Коли None — це не помилка

None $\neq$ Error

None — це валідна інформація про стан системи.

Параметр не задано | Значення ще не обчислено | Запис відсутній

Якщо відсутність є проблемою — використовуйте Result.

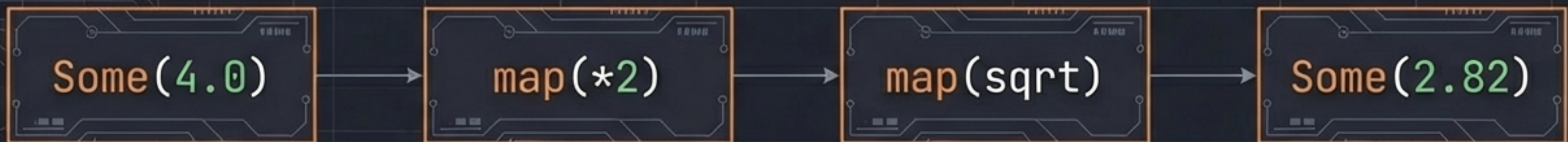
Технічна реалізація Option

```
enum Option<T> {  
    None,  
    Some(T),  
}
```



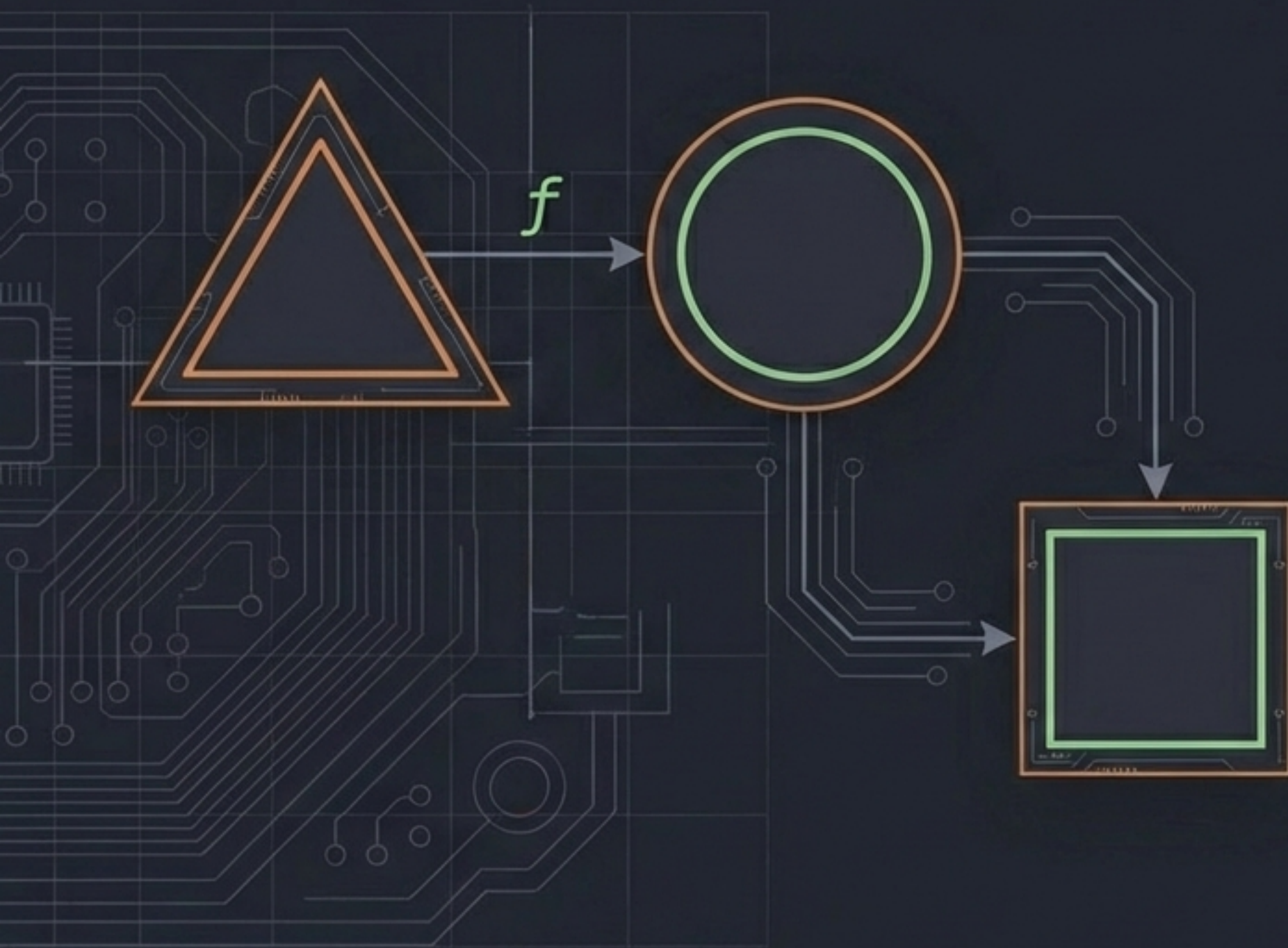
Zero-cost Abstraction.
0x0 address = None.

Монадічна композиція (Map, And_then)



```
let input = Some(4.0);  
  
// Декларативний ланцюжок  
let result = input  
  .map(|x| x * 2.0)           // Some(8.0)  
  .map(|x| x.sqrt())         // Some(2.82...)  
  .unwrap_or(0.0);
```

Алгебраїчні властивості Option



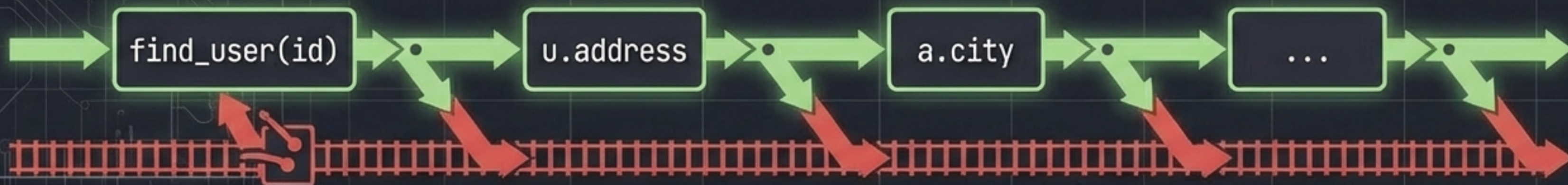
- Satisfies Functor & Monad Laws

1. Асоціативність
2. Pattern Matching

Математична база для надійного коду.

Практика композиції

Success / Some Path



Failure / None Path

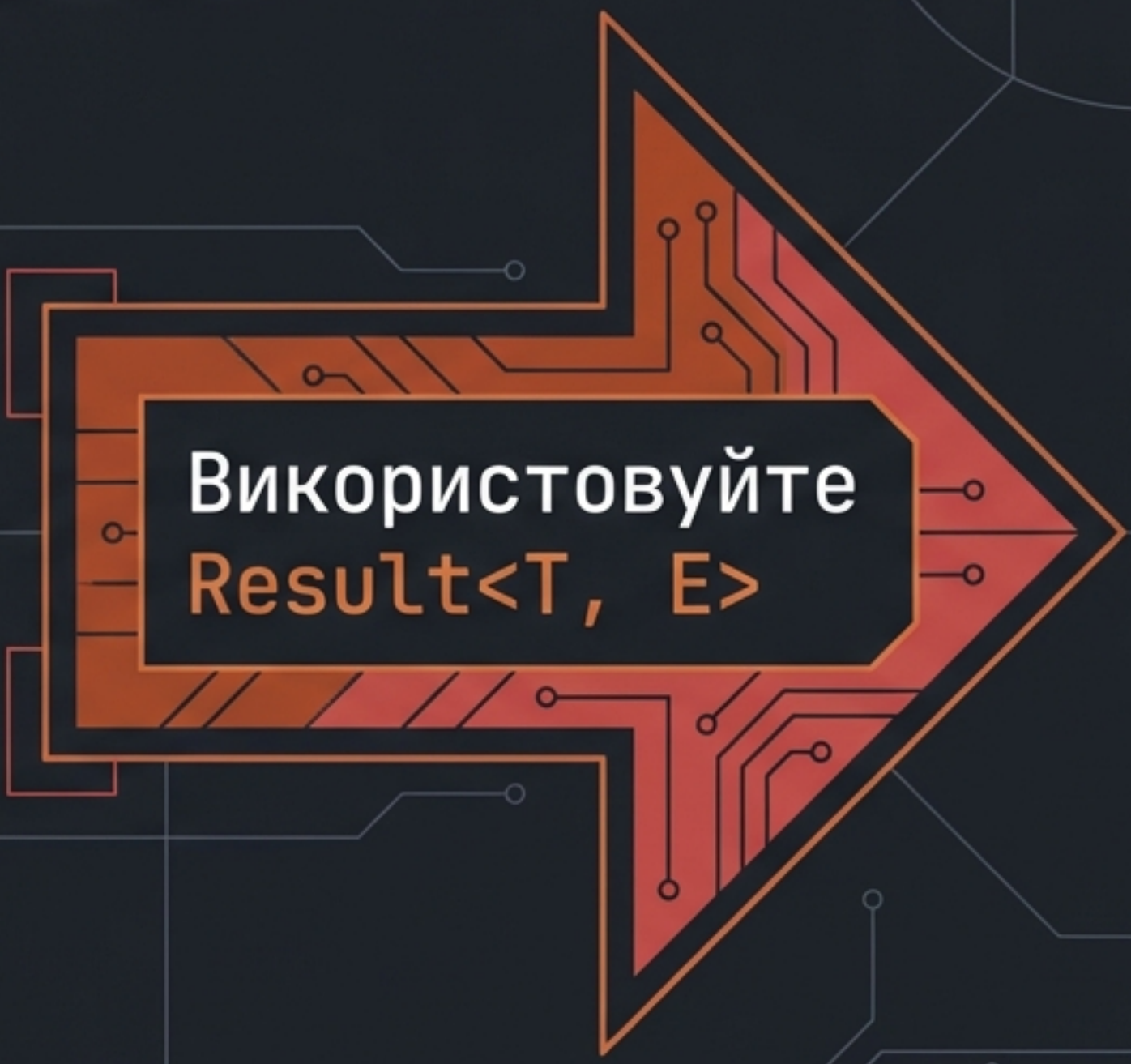
```
fn get_user_city(id: i32) -> Option<String> {  
    find_user(id)           // Option<User>  
    .and_then(|u| u.address) // Option<Address>  
    .map(|a| a.city)        // Option<String>  
}
```

// Short-circuit: ланцюжок переривається автоматично

Межі застосування Option

Коли Option недостатньо?

- ⚠ Потрібна діагностика (Чому?)
- ⚠ Різна логіка для різних причин відсутності
- ⚠ Логування помилок



Використовуйте
`Result<T, E>`

Підсумок: Культура роботи з помилками

1. Помилка – це дані. (Option/Result)
2. Будьте чесними. (Explicit Contracts)
3. Total Functions. (No Panics)
4. Fail-fast. (Не маскуйте дефекти)
5. Антипатерн: Не губіть контекст помилки.

Безпека через дисципліну типів.